

FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

Department of Artificial Intelligence & Data Science · Karachi Campus

PROGRAMMING FOR ARTIFICIAL INTELLIGENCE

Laboratory Manual

LAB 07

NumPy and the N-Dimensional Array

Array Creation · Indexing and Slicing · Views and Copies · Reshaping
Broadcasting · Vectorization · Aggregation by Axis · Linear Algebra

PREPARED BY

Khadeejah Ashraf

Lab Instructor | Department of Artificial Intelligence & Data Science

Contents

1. Lab Objectives
2. Lab Rules and Conduct
3. Why NumPy Exists
4. Creating Arrays
5. The Anatomy of an ndarray
6. Indexing and Slicing
7. Boolean Masks and Fancy Indexing
8. Reshaping and Rearranging
9. Broadcasting
10. Vectorization, ufuncs and Aggregation
11. Random Numbers
12. Linear Algebra Essentials
13. Two Worked Examples
14. Common Errors
15. Quick Reference
16. Class Tasks (to be completed during the session)
17. Home Tasks
18. Submission and Assessment

1. Lab Objectives

By the end of this session, you should be able to:

- Explain why a NumPy array is smaller and faster than a Python list holding the same numbers.
- Create arrays from lists and with `arange`, `linspace`, `zeros`, `ones`, `full`, `eye` and the `*_like` family.
- Read an array's `shape`, `ndim`, `size`, `dtype` and `nbytes`, and say what each one tells you.
- Choose a `dtype` deliberately, and recognise integer overflow and silent truncation when they happen.
- Index and slice one-, two- and three-dimensional arrays, including with negative indices and steps.
- State the difference between a **view** and a **copy**, and predict when a change through one name is visible through another.
- Select elements with boolean masks and with fancy (integer-array) indexing, and combine conditions with `&`, `|` and `~`.
- Reshape, flatten, transpose and add axes with `newaxis`, and join or split arrays along a chosen axis.
- Apply the broadcasting rules to predict the shape of a result — and to explain why a particular pair of shapes is rejected.
- Replace an explicit Python loop with a vectorized expression, and measure the difference.
- Aggregate along `axis=0` and `axis=1`, and use `keepdims` when the result must broadcast back.
- Generate reproducible random data with `np.random.default_rng`, and carry out basic matrix algebra with `@` and `np.linalg`.

2. Lab Rules and Conduct

- Maintain discipline throughout the lab session.
- Listen carefully and follow the instructions as they are given.
- Raise your hand if you face any problem; do not leave your seat.
- **Class tasks** must be completed and demonstrated during the session itself.

- **Home tasks** must be submitted before the deadline announced on the course portal.
- Get your work checked by the instructor before the end of the session.

Reminder: work that is not checked during the session carries no marks, even if the code runs correctly.

3. Why NumPy Exists

Everything you have written so far has stored numbers in Python lists. A list is wonderfully flexible — it can hold anything, and it can grow — but that flexibility is paid for on every single element. Each number in a list is a full Python object with its own header, and the list itself stores only **pointers** to those objects scattered around memory. A **NumPy array** gives up the flexibility and keeps the numbers themselves, all of one type, side by side in one block of memory.

Two consequences follow, and they are the entire reason this library exists.

3.1 Less Memory

```
import sys
import numpy as np

py_list = list(range(1000))
np_arr = np.arange(1000)

print("list :", sys.getsizeof(py_list) + sum(sys.getsizeof(v) for v in py_list),
      "bytes")
print("array:", np_arr.nbytes, "bytes")
```

Output

```
list : 36056 bytes
array: 8000 bytes
```

The same thousand integers occupy roughly four and a half times more memory as a list. On a thousand numbers nobody cares; on the millions of numbers in a dataset or a neural network's weights, it decides whether the work fits in memory at all.

3.2 Much Faster

Because the numbers sit together and share one type, NumPy hands the whole array to a loop written in C — one that never creates a Python object, never checks a type, and can use the processor's ability to work on several numbers at once. The Python loop below and the NumPy expression compute exactly the same thing.

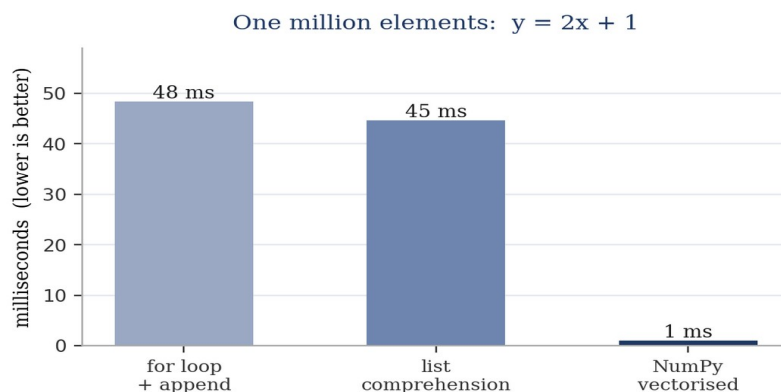


Figure 3.1 The same calculation on one million elements, measured three ways

```
import numpy as np, time

n = 1_000_000
```

```

py = list(range(n))
arr = np.arange(n, dtype=np.float64)

t0 = time.perf_counter()
out = []
for value in py:                # the Python way
    out.append(value * 2.0 + 1.0)
t_loop = time.perf_counter() - t0

t0 = time.perf_counter()
result = arr * 2.0 + 1.0        # the NumPy way - one expression
t_np = time.perf_counter() - t0

print("loop  :", round(t_loop * 1000, 1), "ms")
print("numpy :", round(t_np * 1000, 1), "ms")
print("faster by about", round(t_loop / t_np), "times")

```

Output

```

loop  : 54.5 ms
numpy : 1.0 ms
faster by about 56 times

```

Your numbers will differ. the exact milliseconds depend on the machine and on what else it is doing. The **ratio** is the point, and it grows with the size of the data. This is the single most important idea in the lab: describe the whole array at once and let NumPy do the looping.

3.3 A First Taste

Installation is one command — `pip install numpy` — and the library is imported as `np` by universal convention. Notice in the example below that no loop appears anywhere.

```

import numpy as np

marks = np.array([78, 45, 90, 62])

print(marks * 2)           # every element doubled
print(marks + 5)           # five added to every element
print(marks.mean(), marks.max())
print(marks[marks >= 50])  # only the passing marks

```

Output

```

[156  90 180 124]
[83 50 95 67]
68.75 90
[78 90 62]

```

Watch the difference: with a **list**, `marks * 2` would repeat the list and `marks + 5` would raise a `TypeError`. With an **array**, both operate on the values. Confusing the two is the commonest mistake in this lab.

4. Creating Arrays

4.1 From Python Lists

`np.array()` converts a list into a one-dimensional array, a list of lists into a two-dimensional array, and so on. Every inner list at a given level must have the same length — an array is a rectangle, never a ragged shape.

```

import numpy as np

```

```
a = np.array([1, 2, 3, 4])           # a vector
b = np.array([[1, 2, 3], [4, 5, 6]]) # a matrix
c = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]]) # a tensor

print(a.shape, b.shape, c.shape)
print(a.ndim, b.ndim, c.ndim)
```

Output

```
(4,) (2, 3) (2, 2, 2)
1 2 3
```

Name	Dimensions	Shape looks like	Everyday example
Scalar	0	()	A single temperature
Vector	1	(n,)	One student's marks
Matrix	2	(rows, cols)	A whole class's marks table
Tensor	3 or more	(a, b, c, ...)	A colour image: height × width × 3

4.2 Ranges of Numbers

`arange` is the array version of `range`: start, stop, step, with the stop value excluded. `linspace` instead asks how **many** numbers you want between two endpoints, and includes both ends.

```
print(np.arange(5))           # 0 to 4
print(np.arange(2, 11, 2))     # even numbers, 11 excluded
print(np.linspace(0, 1, 5))    # five values from 0 to 1 inclusive
print(np.arange(0.1, 0.4, 0.1)) # careful - see the note
```

Output

```
[0 1 2 3 4]
[ 2  4  6  8 10]
[0.  0.25 0.5  0.75 1. ]
[0.1 0.2 0.3 0.4]
```

A real trap: the last line asked for values below 0.4 and got four of them, including 0.4 itself. Floating-point steps do not land exactly where you expect, so `arange` cannot always tell when to stop. Whenever the step is fractional, use `linspace` instead.

4.3 Arrays Filled in Advance

When you know the shape you need before you know the values, create the array first and fill it later. The shape is given as a **tuple**.

```
print(np.zeros((2, 3)))        # floats by default
print(np.ones(4, dtype=int))
print(np.full((2, 2), 7))
print(np.eye(3, dtype=int))    # the identity matrix - never hard-code this
print(np.empty(3).shape)       # uninitialised: fast, but contains rubbish
```

Output

```
[[0. 0. 0.]
 [0. 0. 0.]]
[1 1 1 1]
[[7 7]
 [7 7]]
[[1 0 0]]
```

```
[0 1 0]
[0 0 1]]
(3,)
```

The `*_like` family copies the shape and dtype of an array you already have, which saves repeating a shape that may later change.

```
base = np.array([[1, 2, 3], [4, 5, 6]])

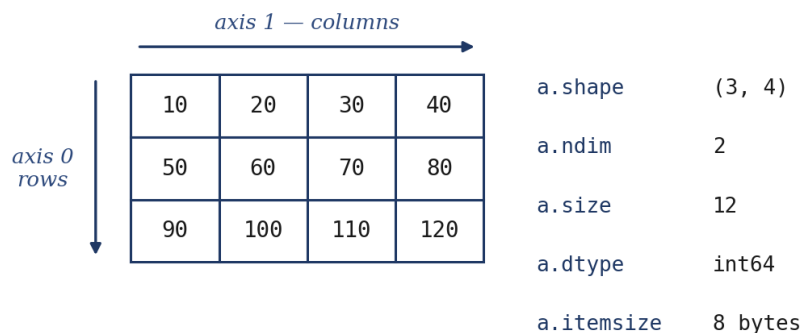
print(np.zeros_like(base))
print(np.ones_like(base).shape)
```

Output

```
[[0 0 0]
 [0 0 0]]
(2, 3)
```

5. The Anatomy of an ndarray

Every array carries a handful of attributes that describe it. Printing them is the first thing to do whenever an operation misbehaves — nine times out of ten the shape is not what you assumed.



one block of memory · every element the same type

Figure 5.1 The attributes of a 3×4 array, and the direction of each axis

```
a = np.array([[10, 20, 30, 40],
              [50, 60, 70, 80],
              [90, 100, 110, 120]])

print("shape   :", a.shape)           # a tuple, one entry per dimension
print("ndim    :", a.ndim)            # how many dimensions
print("size     :", a.size)            # how many elements in total
print("dtype    :", a.dtype)          # the type of every element
print("itemsize:", a.itemsize, "bytes")
print("nbytes   :", a.nbytes, "bytes")
```

Output

```
shape   : (3, 4)
ndim    : 2
size     : 12
dtype    : int64
itemsize: 8 bytes
nbytes   : 96 bytes
```

Axes, once and for all: `axis=0` runs **down** the rows and `axis=1` runs **across** the columns. The shape tuple is in the same order, so in `(3, 4)` the 3 is the length of axis 0. Section 10.3 returns to this.

5.1 Data Types

Every element of an array has the same type, decided when the array is created. NumPy picks the narrowest type that fits everything you gave it, and the choice has consequences.

```
print(np.array([1, 2, 3]).dtype)      # all integers
print(np.array([1.0, 2, 3]).dtype)    # one float makes them all float
print(np.array([1, 2, "three"]).dtype) # one string makes them all strings
print(np.array([1, 2, 3], dtype=np.float32).dtype)

ints = np.array([1, 2, 3])
ints[0] = 9.9                        # stored into an integer array
print(ints)                          # the fraction is silently dropped
```

Output

```
int64
float64
<U21
float32
[9 2 3]
```

dtype	Holds	Bytes each	Typical use
int32 int64	Whole numbers	4 · 8	Counts, indices, labels
float32 float64	Decimals	4 · 8	Measurements; float32 in deep learning
bool	True / False	1	Masks (Section 7)
uint8	0 to 255	1	Image pixels
<U21	Text	varies	Usually a sign that something went wrong

5.2 Overflow and Division

A fixed-width integer type cannot hold a value beyond its range, and NumPy — unlike plain Python — wraps around silently rather than growing the number. Division, on the other hand, always produces floats.

```
small = np.array([120, 125], dtype=np.int8) # int8 holds -128 to 127
print(small + 10)                            # wraps around, no warning

print(np.array([1, 2, 3]) / np.array([2, 2, 2]))
```

Output

```
[-126 -121]
[0.5 1.  1.5]
```

Why this matters later: image pixels are stored as uint8 (0 to 255). Adding brightness to a pixel of 240 gives 24, not 255, unless you convert to a wider type first. Section 13.2 shows the correct pattern.

6. Indexing and Slicing

6.1 One Dimension

A one-dimensional array indexes and slices exactly like a list, with the same `start:stop:step` rules and the same exclusion of the stop value.

```
v = np.arange(10, 60, 10)
print(v)
print(v[0], v[-1])
print(v[1:4])
print(v[::2])
print(v[::-1])
```

Output

```
[10 20 30 40 50]
10 50
[20 30 40]
[10 30 50]
[50 40 30 20 10]
```

6.2 Two Dimensions

Here NumPy departs from nested lists, and for the better. One pair of brackets holds **all** the indices, separated by commas: `a[row, column]`. A slice may be used in either position, and a bare colon means “every one along this axis”.

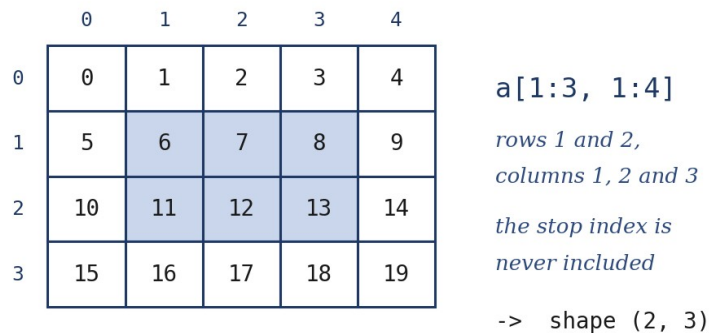


Figure 6.1 `a[1:3, 1:4]` selects rows 1–2 and columns 1–3

```
a = np.arange(20).reshape(4, 5)
print(a)
print(a[1, 3])      # one element: row 1, column 3
print(a[1])         # a whole row
print(a[:, 2])       # a whole column - every row, column 2
print(a[1:3, 1:4])   # a rectangular block
print(a[-1, -1])     # the bottom-right corner
```

Output

```
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]]
8
[5 6 7 8 9]
[ 2  7 12 17]
[[ 6  7  8]
 [11 12 13]]
19
```

Write `a[1, 3]`, not `a[1][3]`: both give the same element, but `a[1][3]` builds a whole temporary row first and then indexes it. The comma form goes straight to the element, and it is the only form that accepts slices on both axes at once.

6.3 Three Dimensions

The pattern continues: one index per axis, in shape order. For a tensor of shape (2, 2, 3) the first index chooses the block, the second the row within it, the third the element within that row.

```
t = np.arange(12).reshape(2, 2, 3)
print(t)
print(t[0, 1, 2])      # block 0, row 1, element 2
print(t.shape)
```

Output

```
[[[ 0  1  2]
  [ 3  4  5]]

 [[ 6  7  8]
  [ 9 10 11]]]
5
(2, 2, 3)
```

6.4 Views and Copies

This is the section students most often skip and most often regret. **Slicing an array does not copy it.** The slice is a ***view***: a second window onto the very same memory. Writing through the view changes the original.

```
a = np.arange(10)
s = a[2:5]          # a view, not a copy
s[0] = 999

print(a)            # the original has changed
print(s.base is a)  # the view remembers what it looks at
```

Output

```
[ 0  1 999  3  4  5  6  7  8  9]
True
```

This is deliberate and valuable — it is how NumPy avoids copying gigabytes of data for every slice. When you genuinely need an independent array, ask for one with `.copy()`.

```
a = np.arange(10)
s = a[2:5].copy()   # an independent array
s[0] = 999

print(a)            # untouched
print(s)
```

Output

```
[0 1 2 3 4 5 6 7 8 9]
[999  3  4]
```

Operation	Gives you
Basic slicing — <code>a[2:5]</code> , <code>a[:, 1]</code> , <code>a.T</code> , <code>a.reshape(...)</code> , <code>a.ravel()</code>	A view — shares memory with the original
Boolean masking — <code>a[a > 5]</code>	A copy — a new array
Fancy indexing — <code>a[[0, 2, 4]]</code>	A copy — a new array
<code>a.copy()</code> , <code>a.flatten()</code> , arithmetic such as <code>a * 2</code>	A copy — a new array

Recognising the symptom: if an array you never assigned to has mysteriously changed, look for a slice of it that you wrote to earlier. This is the same aliasing problem you met with lists in Lab 03, in a faster and less obvious form.

7. Boolean Masks and Fancy Indexing

7.1 Boolean Masks

Comparing an array with a value does not give one answer — it gives an array of `True` and `False`, one per element. That mask can then be used as an index, and only the elements aligned with `True` are selected. This replaces the filtering loop you would otherwise write, and it is the idiom you will use most often in the rest of this course.

```
marks = np.array([78, 45, 90, 62, 33, 88])

mask = marks >= 50
print(mask)
print(marks[mask])                # select using the mask
print(marks[(marks >= 50) & (marks < 85)]) # two conditions
print(mask.sum(), "passed out of", marks.size)
print(np.any(marks > 85), np.all(marks > 30))
```

Output

```
[ True False  True  True False  True]
[78 90 62 88]
[78 62]
4 passed out of 6
True True
```

Use `&`, `|`, `~` — never `and`, `or`, `not`: the Python keywords demand a single true-or-false answer and an array cannot give one, so they raise `ValueError`. And because `&` binds more tightly than `>=`, **every** condition must be wrapped in its own parentheses.

A mask may also be used on the left of an assignment, which changes only the selected elements in place.

```
marks = np.array([78, 45, 90, 62, 33, 88])
marks[marks < 50] = 50          # lift every failing mark to 50
print(marks)
```

Output

```
[78 50 90 62 50 88]
```

7.2 Fancy Indexing

Indexing with a **list of positions** selects exactly those elements, in exactly that order, and may repeat them. For a two-dimensional array, two index lists are paired up element by element.

```
v = np.array([10, 20, 30, 40, 50])
print(v[[0, 2, 4]])          # pick three positions
print(v[[4, 4, 0]])          # order and repetition are allowed

a = np.arange(20).reshape(4, 5)
print(a[[0, 2], [1, 3]])      # the elements (0,1) and (2,3)
print(a[[0, 2]])              # rows 0 and 2, whole
```

Output

```
[10 30 50]
[50 50 10]
[ 1 13]
```

```
[[ 0  1  2  3  4]
 [10 11 12 13 14]]
```

7.3 np.where

`np.where(condition, if_true, if_false)` is the array-wide version of the conditional expression from Lab 02. Given only a condition, it returns the **positions** where that condition holds.

```
marks = np.array([78, 45, 90, 62])

print(np.where(marks >= 50, "pass", "fail"))
print(np.where(marks >= 50, marks, 0))      # keep, or replace with zero
print(np.where(marks >= 50)[0])            # the positions themselves
```

Output

```
['pass' 'fail' 'pass' 'pass']
[78  0 90 62]
[0 2 3]
```

8. Reshaping and Rearranging

8.1 reshape

`reshape` reinterprets the same elements under a new shape. The number of elements must be unchanged, and a single `-1` asks NumPy to work out that dimension for you.

```
a = np.arange(12)
print(a.reshape(3, 4))
print(a.reshape(2, 6).shape)
print(a.reshape(3, -1).shape)    # -1 means "whatever fits"
print(a.reshape(2, 2, 3).shape)
```

Output

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
(2, 6)
(3, 4)
(2, 2, 3)
```

8.2 Flattening

`ravel` and `flatten` both return a one-dimensional version, but `ravel` gives a **view** where it can and `flatten` always copies. The distinction is exactly the one from Section 6.4.

```
a = np.arange(6).reshape(2, 3)
print(a.ravel())
print(a.flatten())

r = a.ravel()
r[0] = 99      # writing through the view...
print(a)       # ...changes the original
```

Output

```
[0 1 2 3 4 5]
[0 1 2 3 4 5]
[[99  1  2]
 [ 3  4  5]]
```

8.3 Transpose and New Axes

.T swaps the axes of a matrix. `np.newaxis` (or `None`) inserts an axis of length 1, which is how a flat vector is turned into a genuine column or row — the move that makes broadcasting work in the next section.

```
a = np.arange(6).reshape(2, 3)
print(a.T)
print(a.T.shape)

v = np.array([1, 2, 3])
print(v.shape, v[:, np.newaxis].shape, v[np.newaxis, :].shape)
print(v[:, np.newaxis])
```

Output

```
[[0 3]
 [1 4]
 [2 5]]
(3, 2)
(3,) (3, 1) (1, 3)
[[1]
 [2]
 [3]]
```

A vector is neither a row nor a column: shape (3,) has only one axis. It is not (1, 3) and not (3, 1). Most shape complaints in this lab come from assuming otherwise, and `newaxis` is the fix.

8.4 Joining and Splitting

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

print(np.concatenate([a, b], axis=0))      # stack rows
print(np.concatenate([a, b], axis=1))      # stack columns
print(np.vstack([a, b]).shape, np.hstack([a, b]).shape)
print(np.stack([a, b]).shape)              # a NEW axis: (2, 2, 2)
print(np.split(np.arange(9), 3))
```

Output

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
[[1 2 5 6]
 [3 4 7 8]]
(4, 2) (2, 4)
(2, 2, 2)
[array([0, 1, 2]), array([3, 4, 5]), array([6, 7, 8])]
```

Function	Effect
<code>np.concatenate([a, b], axis=n)</code>	Joins along an axis that already exists
<code>np.vstack</code> / <code>np.hstack</code>	Shorthand for <code>axis=0</code> / <code>axis=1</code>
<code>np.stack([a, b])</code>	Creates a new axis — the result has one more dimension
<code>np.split(a, n)</code> / <code>np.array_split(a, n)</code>	Cuts into <code>n</code> pieces; <code>array_split</code> allows uneven ones

Arrays do not grow: there is no `append` that adds to an array in place. `np.append` and `np.concatenate` build a whole new array every time, so doing either inside a loop is far slower than building a Python list and converting it once at the end.

9. Broadcasting

Broadcasting is the rule that lets NumPy combine arrays of different shapes. You have already used it without noticing: `marks * 2` works because the single number 2 is stretched to match every element. The same idea extends to whole rows and columns, and it is what makes most loops unnecessary.

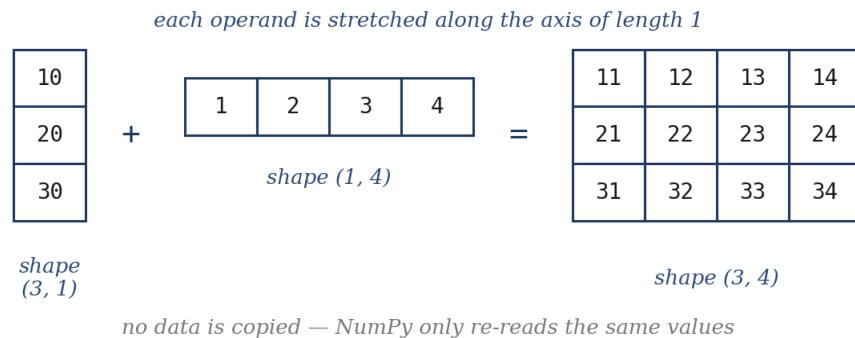


Figure 9.1 A (3, 1) column and a (1, 4) row broadcast to a (3, 4) result

9.1 The Rules

NumPy compares the two shapes from the **right-hand end**, one axis at a time. For each pair of lengths:

- If they are equal, that axis is fine.
- If one of them is 1, that operand is stretched along the axis to match the other.
- Otherwise the shapes are incompatible and a `ValueError` is raised.

A shape that has fewer axes than the other is treated as though it had 1s on the left. Working through one example line by line:

```
a.shape      (3, 1)
b.shape      (4,)    ->  treated as (1, 4)
-----
compare      1 vs 4  ->  stretch a to 4
              3 vs 1  ->  stretch b to 3
result       (3, 4)
```

9.2 Broadcasting in Practice

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a * 10)          # a scalar broadcasts to everything
print(a - a.mean())    # centre the whole array on zero
```

Output

```
[[10 20 30]
 [40 50 60]]
[[-2.5 -1.5 -0.5]
 [ 0.5  1.5  2.5]]
```

```
col = np.array([[10], [20], [30]])    # shape (3, 1)
row = np.array([1, 2, 3, 4])          # shape (4,)
```

```
print(col.shape, row.shape)
print(col + row)                # -> shape (3, 4)
```

Output

```
(3, 1) (4,)
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
```

The genuinely useful case is applying one value per column, or per row, across a whole table. Here three subjects carry different weights, and each column is then centred on its own mean — two loops that never had to be written.

```
marks = np.array([[78, 45, 90],
                  [88, 92, 95],
                  [40, 38, 52]])
weights = np.array([0.3, 0.3, 0.4])    # one weight per subject

print(marks * weights)                 # (3,3) * (3,) -> per column
print((marks * weights).sum(axis=1))    # each student's weighted total
print(marks - marks.mean(axis=0))       # centre each subject
```

Output

```
[[23.4 13.5 36. ]
 [26.4 27.6 38. ]
 [12.  11.4 20.8]]
[72.9 92.  44.2]
[[ 9.33333333 -13.33333333  11.      ]
 [ 19.33333333  33.66666667  16.      ]
 [-28.66666667 -20.33333333 -27.     ]]
```

9.3 When Broadcasting Refuses

Read the error message: it prints both shapes, and the mismatched axis is always visible in it. The cure is nearly always to insert an axis so that the alignment comes out right.

```
a = np.ones((3, 4))
b = np.ones((3,))

try:
    print((a + b).shape)    # (3,4) vs (3,) -> compares 4 with 3
except ValueError as err:
    print("ValueError:", err)

try:
    a + np.ones((4, 3))
except ValueError as err:
    print("ValueError:", err)

print((a + b[:, np.newaxis]).shape)    # (3,4) vs (3,1) -> works
```

Output

```
ValueError: operands could not be broadcast together with shapes (3,4) (3,)
ValueError: operands could not be broadcast together with shapes (3,4) (4,3)
(3, 4)
```

10. Vectorization, ufuncs and Aggregation

Vectorization means expressing a calculation over a whole array at once instead of element by element. Section 3.2 measured why. This section is about how.

10.1 Universal Functions

A **ufunc** is a function that applies element by element to a whole array and returns a new array. NumPy provides one for every mathematical operation you are likely to want, and they are the building blocks of every vectorized expression.

```
a = np.array([1, 4, 9, 16])

print(np.sqrt(a))
print(np.exp([0, 1]))
print(np.round(np.log(a), 3))
print(np.abs([-3, 3]))
print(np.maximum([1, 5, 3], [4, 2, 3]))    # element-by-element larger
```

Output

```
[1.  2.  3.  4.]
[1.         2.71828183]
[0.         1.386  2.197  2.773]
[3  3]
[4  5  3]
```

Family	Members
Arithmetic	+ - * / // % **, and np.add, np.multiply, ...
Mathematical	np.sqrt, np.exp, np.log, np.sin, np.cos, np.abs
Rounding	np.round, np.floor, np.ceil, np.clip
Comparison	> >= == !=, np.maximum, np.minimum
Logical	& ~, np.logical_and, np.any, np.all

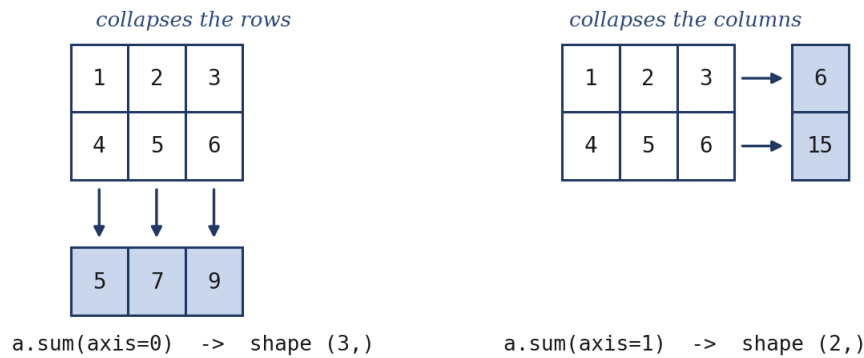
10.2 Turning a Loop into an Expression

The recipe is always the same: find the loop, ask what it does to **every** element, and write that as one expression. Three typical translations:

Instead of a loop that...	Write
Applies a formula to each element	result = 2 * a + 1
Keeps the elements passing a test	result = a[a > 50]
Chooses between two values per element	result = np.where(a > 50, a, 0)
Adds up a running total	total = a.sum()
Compares two lists position by position	result = a * b or a @ b

10.3 Aggregation and the axis Argument

An aggregation such as `sum` or `mean` collapses an array to fewer numbers. With no argument it collapses everything to one value. With `axis=` it collapses only that axis — and the rule to remember is that **the named axis disappears from the shape**.

Figure 10.1 *axis=0 collapses the rows; axis=1 collapses the columns*

```
a = np.array([[1, 2, 3], [4, 5, 6]])

print(a.sum(), a.mean(), a.max())    # the whole array
print(a.sum(axis=0))                 # down the rows -> one per column
print(a.sum(axis=1))                 # across columns -> one per row
print(a.mean(axis=1, keepdims=True).shape)

print(a.argmax())                    # position in the flattened array
row, col = np.unravel_index(a.argmax(), a.shape)
print("largest value is at row", row, "column", col)
```

Output

```
21 3.5 6
[5 7 9]
[ 6 15]
(2, 1)
5
largest value is at row 1 column 2
```

What keepdims is for: `a.mean(axis=1)` has shape `(2,)`, which will not broadcast back against `(2, 3)`. With `keepdims=True` the result is `(2, 1)`, which broadcasts perfectly — exactly what row-wise centring needs.

10.4 Statistics and Sorting

```
data = np.array([12, 7, 40, 3, 25, 18])

print(data.mean(), data.std().round(3), data.var().round(3))
print(np.median(data), data.min(), data.max())
print(np.sort(data))           # a sorted copy
print(np.argsort(data))        # the positions that would sort it
print(np.cumsum(data))         # running total
```

Output

```
17.5 12.339 152.25
15.0 3 40
[ 3  7 12 18 25 40]
[3 1 0 5 4 2]
[ 12  19  59  62  87 105]
```

11. Random Numbers

Random data is how you test code before real data exists, and how models are initialised. Modern NumPy asks you to create a **generator** first, rather than calling `np.random.rand` directly; the generator keeps its own state, which makes results reproducible and keeps separate parts of a program from interfering with one another.

```
rng = np.random.default_rng(seed=42)      # one generator, one seed

print(rng.integers(1, 7, size=5))          # five dice rolls, 7 excluded
print(rng.random(3).round(3))             # uniform between 0 and 1
print(rng.normal(loc=70, scale=10, size=4).round(2)) # a bell curve
print(rng.choice([2, 5, 9, 10], size=(2, 2))) # pick from a set
```

Output

```
[1 5 4 3 3]
[0.697 0.094 0.976]
[71.28 66.84 69.83 61.47]
[[10  9]
 [ 5  2]]
```

The **seed** is what makes an experiment repeatable: the same seed always produces the same sequence, on any machine. Every experiment you report in this course should set one.

```
a = np.random.default_rng(0).integers(0, 100, 5)
b = np.random.default_rng(0).integers(0, 100, 5)

print(a)
print(b)
print(np.array_equal(a, b))
```

Output

```
[85 63 51 26 30]
[85 63 51 26 30]
True
```

```
rng = np.random.default_rng(7)

deck = np.arange(10)
rng.shuffle(deck)      # shuffles in place
print(deck)
print(rng.permutation(5)) # returns a shuffled copy
```

Output

```
[8 0 7 1 3 6 2 4 5 9]
[4 3 1 2 0]
```

12. Linear Algebra Essentials

The distinction to fix in your mind is that `*` multiplies element by element, while `@` performs true matrix multiplication. They are different operations, and only one of them cares that the inner dimensions match.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print(A * B)      # element by element
print(A @ B)      # matrix product
print(np.dot(A, B)) # the same as @

M = np.arange(6).reshape(2, 3)
N = np.arange(6).reshape(3, 2)
print((M @ N).shape) # (2,3) @ (3,2) -> (2,2)
```

Output

```
[[ 5 12]
 [21 32]]
```

```
[[19 22]
 [43 50]]
[[19 22]
 [43 50]]
(2, 2)
```

The matrix-product rule: $(m, n) @ (n, p)$ gives (m, p) . The two inner numbers must be equal; if they are not, NumPy reports exactly which pair failed.

The `np.linalg` submodule holds the standard operations of a linear algebra course.

```
A = np.array([[4.0, 7.0], [2.0, 6.0]])

print(np.linalg.det(A).round(3))          # determinant
print(np.linalg.inv(A).round(3))          # inverse
print((A @ np.linalg.inv(A)).round(10))   # a matrix times its inverse

vals, vecs = np.linalg.eig(np.array([[2.0, 0.0], [0.0, 3.0]]))
print(vals)

coef = np.array([[2.0, 1.0], [1.0, 3.0]]) # 2x + y = 8
rhs = np.array([8.0, 13.0])              # x + 3y = 13
print(np.linalg.solve(coef, rhs))
```

Output

```
10.0
[[ 0.6 -0.7]
 [-0.2  0.4]]
[[ 1. -0.]
 [-0.  1.]]
[2. 3.]
[2.2 3.6]
```

Function	Returns
<code>A @ B · np.dot · np.matmul</code>	The matrix product
<code>A.T</code>	The transpose
<code>np.linalg.det(A)</code>	The determinant (square matrices only)
<code>np.linalg.inv(A)</code>	The inverse — raises if the matrix is singular
<code>np.linalg.eig(A)</code>	Eigenvalues and eigenvectors
<code>np.linalg.solve(A, b)</code>	The solution of $Ax = b$, faster and safer than <code>inv(A) @ b</code>
<code>np.cross(a, b)</code>	The cross product of two 3-element vectors

13. Two Worked Examples

13.1 Standardising a Dataset

Standardising means shifting each column to a mean of zero and scaling it to a standard deviation of one — the step almost every machine-learning model expects, and the one you will meet again in Lab 10. Two loops would be needed in plain Python; broadcasting removes both.

```
rng = np.random.default_rng(1)
```

```
data = rng.integers(30, 100, size=(6, 3))      # 6 students, 3 subjects
print(data)
print("column means:", data.mean(axis=0).round(2))

standard = (data - data.mean(axis=0)) / data.std(axis=0)
print(standard.round(2))

print("new means:", standard.mean(axis=0).round(10))
print("new stds :", standard.std(axis=0).round(10))
```

Output

```
[[63 65 82]
 [96 32 40]
 [87 96 47]
 [51 90 59]
 [49 87 47]
 [58 75 68]]
column means: [67.33 74.17 57.17]
[[-0.24 -0.43  1.73]
 [ 1.6  -1.97 -1.19]
 [ 1.1   1.02 -0.71]
 [-0.91  0.74  0.13]
 [-1.03  0.6  -0.71]
 [-0.52  0.04  0.75]]
new means: [ 0. -0.  0.]
new stds : [1. 1. 1.]
```

The expression `(data - data.mean(axis=0)) / data.std(axis=0)` is `(6,3)` combined with `(3,)` twice, so each column is matched with its own mean and its own standard deviation. Written as loops it would be a dozen lines; here it is one, and it is faster.

13.2 An Image as a Tensor

A colour image is a three-dimensional array of shape `(height, width, 3)`, holding red, green and blue for every pixel as `uint8`. Everything in this manual applies to it directly, which is why Lab 14 can treat image processing as ordinary array work.

```
rng = np.random.default_rng(3)
image = rng.integers(0, 256, size=(4, 4, 3), dtype=np.uint8)
print(image.shape, image.dtype, image.nbytes, "bytes")

grey = image.mean(axis=2)      # average the colour axis away
print(grey.round(1))

print(image[:2, :2, 0])       # the red channel of the top-left corner

# brighten safely: widen the type first, clip, then narrow again
brighter = np.clip(image.astype(np.int16) + 40, 0, 255).astype(np.uint8)
print(brighter[0, 0], image[0, 0])
```

Output

```
(4, 4, 3) uint8 48 bytes
[[210.7 126.7 167.7 165. ]
 [107.  116.7 118.  106.3]
 [ 59.  141.7  57.   15.7]
 [125.3  24.   51.7 146.3]]
[[248 207]
 [ 5  60]]
[255 234 230] [248 194 190]
```

Why the widening matters: the first pixel's red was 248. Adding 40 directly in `uint8` would wrap it around to 32 — a bright pixel turning almost black. Converting to `int16`, clipping at 255 and converting back gives the correct 255.

14. Common Errors

```
try:
    np.arange(12).reshape(5, 3)
except ValueError as err:
    print("ValueError:", err)

try:
    if np.array([1, 2, 3]) > 2:      # an array is not one true-or-false value
        pass
except ValueError as err:
    print("ValueError:", err)
```

Output

```
ValueError: cannot reshape array of size 12 into shape (5,3)
ValueError: The truth value of an array with more than one element is ambiguous. Use
a.any() or a.all()
```

Message or symptom	Usual cause
ValueError: operands could not be broadcast together	The shapes do not align from the right; print both shapes and insert an axis.
ValueError: cannot reshape array of size 12 into shape (5,3)	The element count must be unchanged — 5×3 is not 12.
ValueError: The truth value of an array ... is ambiguous	An array was used in an <code>if</code> , <code>or</code> and / <code>or</code> was used instead of <code>&</code> / <code> </code> .
IndexError: too many indices for array	More indices than the array has dimensions — check <code>a.ndim</code> .
An array changed that you never assigned to	You wrote through a view ; take a <code>.copy()</code> (Section 6.4).
A fractional value became a whole number	The array's <code>dtype</code> is an integer type; create it with <code>dtype=float</code> .
Results wrap around to negative numbers	Integer overflow in a narrow <code>dtype</code> such as <code>int8</code> or <code>uint8</code> .
TypeError: unsupported operand type(s)	One operand is still a Python list; convert with <code>np.array()</code> .
<code>np.dot</code> complains about aligned dimensions	The inner dimensions differ — $(m,n) @ (n,p)$ is the rule.
The code is correct but painfully slow	A Python loop is still in there, or an array is being grown with <code>np.append</code> inside a loop.

15. Quick Reference

Form	Meaning	Example
<code>np.array(L)</code>	Build an array from a list	<code>np.array([[1,2],[3,4]])</code>

Form	Meaning	Example
<code>np.arange(a, b, s)</code>	Range with a step — <code>b</code> excluded	<code>np.arange(2, 11, 2)</code>
<code>np.linspace(a, b, n)</code>	<code>n</code> values including both ends	<code>np.linspace(0, 1, 5)</code>
<code>zeros ones full eye</code>	Pre-filled arrays; <code>eye</code> is the identity	<code>np.eye(3)</code>
<code>a.shape ndim size dtype</code>	Describe the array	Print these when debugging
<code>a[i, j]</code> <code>a[1:3, 1:4]</code>	Index and slice — one bracket, commas	<code>a[:, 2]</code> is a whole column
<code>a[a > 50]</code>	Boolean mask — returns a copy	<code>a[(a>50) & (a<85)]</code>
<code>a[[0, 2, 4]]</code>	Fancy indexing — returns a copy	Order and repeats allowed
<code>np.where(c, x, y)</code>	Element-wise choice	<code>np.where(a>50, a, 0)</code>
<code>a.reshape(r, -1)</code>	New shape; <code>-1</code> is inferred	Element count must match
<code>a.ravel()</code> <code>a.flatten()</code>	Flatten — view · copy	<code>a.T</code> transposes
<code>a[:, np.newaxis]</code>	Turn <code>(n,)</code> into a column <code>(n,1)</code>	The usual broadcasting fix
<code>concatenate vstack stack</code>	Join along an axis / a new axis	<code>np.split(a, 3)</code> cuts up
<code>a.sum(axis=0)</code>	The named axis disappears	<code>keepdims=True</code> keeps it as 1
<code>a * b</code> vs <code>a @ b</code>	Element-wise vs matrix product	<code>(m,n) @ (n,p) -> (m,p)</code>
<code>np.linalg.*</code>	<code>det</code> , <code>inv</code> , <code>eig</code> , <code>solve</code>	<code>np.linalg.solve(A, b)</code>
<code>np.random.default_rng(42)</code>	A seeded generator	<code>rng.integers</code> , <code>rng.normal</code>
<code>a.copy()</code>	An independent array	Slices are views, not copies

16. Class Tasks

These five tasks are to be completed **during the lab session** and demonstrated to the instructor before you leave. Work in the accompanying notebook `Lab07_NumPy.ipynb`, in the section titled **Class Tasks**. Unless a task says otherwise, **no Python for loop may appear in your solution** — that restriction is the whole point of the lab. Print the shape and dtype of every array you create.

C1. Array Workshop — creation, attributes and dtypes. Build each of the following without writing out the values by hand: a vector of the even integers strictly between 20 and 50; a 3×3 matrix of the odd numbers from 1 up to but not including 19; ten evenly spaced values from 0 to 2 inclusive; a 4×4 identity matrix; and a $2 \times 3 \times 4$ tensor of zeros. For each one print the array, its shape, ndim, size, dtype and nbytes.

Then demonstrate three dtype behaviours with a short comment explaining each: that assigning `9.9` into an integer array stores 9; that `np.array([120, 125], dtype=np.int8) + 10` wraps around to negatives; and that mixing a string into a list of numbers makes the whole array textual. Finally, compare the memory used by `list(range(10000))` and `np.arange(10000)` and print the ratio.

C2. The Marks Table — indexing, slicing, views and masks. Using a seeded generator (`np.random.default_rng(42)`), create a 6×4 integer array of marks between 30 and 100 representing six students and four subjects. Without any loop, print: the whole third student's row; the second subject's column; the 3×2 block formed by the first three students and the last two subjects; every mark of 50 or more; the

marks that are between 60 and 85; how many marks are failing; and the array with every failing mark lifted to exactly 50 — leaving the original unchanged.

Then prove the view-versus-copy rule in code. Take a slice of the first two rows, write a 999 into it, and print the original to show it changed; repeat with `.copy()` and show that it did not. State in a comment which of a boolean mask, a basic slice and fancy indexing give a view and which give a copy.

C3. Reshaping and Broadcasting Drills. Start from `np.arange(24)`. Reshape it into (4, 6), then (2, 3, 4), then (6, -1), printing each shape. Flatten one of them with both `ravel` and `flatten`, and show in code that only one of the two is a view. Transpose the (4, 6) matrix and confirm the shape.

For the broadcasting half, predict **on paper first** and then verify: (4, 6) with (6,); (4, 6) with (4,); (4, 6) with (4, 1); (3, 1) with (1, 5). For each pair write the result shape, or the exact `ValueError` message if it is rejected, and repair the failing case with `np.newaxis`. Finish by using broadcasting — and no loop — to multiply each column of a 4×3 marks table by the weights [0.25, 0.35, 0.40] and print each student's weighted total.

C4. Vectorization Contest — loops against arrays. For an array of one million floats, write each of the following twice, once with a Python loop and once as a single NumPy expression, timing both with `time.perf_counter()` and printing the speed-up: (i) $y = 3x^2 + 2x + 1$; (ii) the count of elements greater than 0.5; (iii) the sum of the squares of only the elements above the mean; (iv) replacing every negative value with zero. Verify with `np.allclose` that each pair of results actually agrees.

Present the four speed-ups in a small printed table, and write two sentences in a comment explaining **why** the NumPy version is faster — your explanation must mention the type of the elements and where the loop actually runs.

C5. A Class Result Sheet — the whole lab in one program. With a seeded generator, build a marks table for 10 students and 5 subjects (values 0–100) and the accompanying list of subject names. Without any loop, compute and print: every student's total and average; every subject's mean, highest and lowest mark; the standardised table where each subject has a mean of 0 and a standard deviation of 1; a letter grade for every mark using `np.where` (or nested `np.where`) rather than `if`; the roll number of the topper found with `argmax`; and the name of the hardest subject, that is the one with the lowest mean.

Then add a 5 % bonus to the weakest subject only, using broadcasting with a weights vector rather than a loop, and use `np.clip` to make sure no mark exceeds 100. Print the table before and after. Finish by saving the final table to a CSV file with `np.savetxt` and loading it back with `np.loadtxt`, showing with `np.allclose` that the round trip is exact.

Assessment of class tasks: you must be able to explain any line of your code — in particular the shape of every intermediate result, why a given operation broadcasts, and whether a name refers to a view or a copy. A working program that you cannot explain earns no marks.

17. Home Tasks

Attempt all the questions below. Write each program in a **separate .py file**, print the shape of every array you create, and submit according to the naming convention in Section 18. Avoid Python loops wherever an array operation will do the same work.

- Q1.** Write a program to create a NumPy vector of all the even integers from 20 to 50, excluding both 20 and 50. Print the vector, its length and its sum.
- Q2.** Create a multi-dimensional array of 3 rows and 3 columns holding the odd numbers from 1 to 19, including 1 and excluding 19. Then print every element of it — once using `nditer` and once using a single vectorized operation instead of iterating — and explain the difference in a comment.
- Q3.** Create a 3×3 matrix containing only even numbers. Multiply every element of it by 4, and then multiply the resulting matrix by a 3×3 identity matrix. The identity matrix must not be hard-coded. Show that the final result equals the intermediate one, and explain in a comment why.

- Q4.** Generate a 4×4 matrix whose entries are drawn at random only from the values 2, 5, 9 and 10, multiply it by the identity matrix, and then add the result to any 4×4 matrix containing only odd numbers. Use a seeded generator so that your output can be reproduced.
- Q5.** Write a program to multiply a 5×3 matrix by a 3×2 matrix. Print the shape of the result, and then show what `*` does to two matrices of the same shape, explaining the difference between the two operators in a comment.
- Q6.** Create a random array of 1000 elements and compute its average, variance and standard deviation. Save all three results to a text file created at run time, then read the file back and print its contents.
- Q7.** Create a structured array from a set of student names, heights and classes, with appropriate data types. Sort it by class, and by height where two students share a class. Print the array before and after sorting.
- Q8.** Given a 6×6 matrix of random integers between 1 and 100, use slicing and boolean masks — and no loop — to print: the four corner values; the border elements only; every element on the main diagonal; every even element; and the matrix with every element above 50 replaced by 0.
- Q9.** Write a program that normalises a 5×4 matrix of random values twice: once so that every **column** has a mean of 0 and a standard deviation of 1, and once so that every **row** does. Use broadcasting with `keepdims` and verify both results by printing the new means and standard deviations.
- Q10.** Time the calculation of the element-wise product of two sequences of 500,000 numbers, once with a Python loop over two lists and once with NumPy arrays. Report both times and the speed-up, and confirm with `np.allclose` that the two results agree.
- Q11.** Represent a small greyscale image as an 8×8 array of `uint8` values. Without any loop, produce and print: the image flipped upside down; flipped left to right; rotated by 90 degrees; brightened by 50 without wrapping around (use widening and `np.clip`); and converted to pure black and white by a threshold of 128.
- Q12.** Solve the system of linear equations $3x + 2y - z = 10$, $2x - 2y + 4z = -2$, $-x + 0.5y - z = 0$ using `np.linalg.solve`. Verify your answer by substituting it back with the `@` operator, print the determinant and the inverse of the coefficient matrix, and state in a comment what a determinant of zero would have meant.

18. Submission and Assessment

Follow the naming convention below so that your work can be identified and graded correctly:

- **Class tasks:** completed in the notebook `RollNo_Lab07_ClassTasks.ipynb` and demonstrated before the end of the session. Run **Kernel ► Restart & Run All** before you call the instructor.
- **Home tasks:** one script per question, named `RollNo_Lab07_Qn.py` — for example `22K-1234_Lab07_Q8.py`.
- Every program that uses random data must set a seed, so that your output can be reproduced during marking.
- Place everything for this lab in a single folder and compress it before uploading.
- Every file must begin with a comment giving your name, roll number and the question number.
- Code that does not run will not be marked. Test your program before calling the instructor.

Assessment Criterion	Weight
Correctness of output, including the shape and dtype of every result	30%
Class tasks completed and demonstrated in the session	20%
Vectorized solutions — array operations used instead of Python loops	25%
Correct use of indexing, broadcasting and the axis argument	15%
Readability: naming, comments, reproducible seeds	10%

End of Lab 07